MANUAL PASO A PASO CON TDD

Rally de la Independencia AR

Juego de realidad aumentada para recorrer Dolores Hidalgo

Radar · Frío o caliente · Trivia histórica · Álbum coleccionable · Modo demo

Desarrollo Móvil Integral · UTNG

Prof. Anastacio Rodríguez García

Antes de empezar: ¿qué vamos a construir?

Es una tarde de domingo en el Jardín Principal de Dolores Hidalgo. Un grupo de estudiantes camina con el teléfono en alto. En la pantalla, sobre la imagen de la cámara, flota un círculo con una campana: es Miguel Hidalgo, escondido a 180 metros. Un radar en la esquina muestra un punto naranja; un letrero dice "🔥 Caliente". Al acercarse a menos de 15 metros, el círculo brilla en verde: "¡Tócame!". Tocan, aparece una pregunta de historia, la responden bien y el personaje entra a su álbum. Quedan cinco más por encontrar.

Eso construirás: un juego de búsqueda con realidad aumentada (RA) basado en ubicación, que convierte la historia de la Independencia en una aventura al aire libre.

¿En qué se diferencia de la guía "Dolores AR"?

Si ya hiciste el manual de Dolores AR (guía turística con etiquetas), este es su "hermano de juego". Aquí comparten la idea de usar GPS y brújula, pero la meta cambia por completo y aparecen conceptos nuevos:

- Los objetivos no están en lugares fijos: se generan al azar alrededor de un centro, con un generador de números aleatorios inyectado para poder probarlo.
- Hay reglas de juego (puntos, intentos, huida del personaje) modeladas con una jerarquía "sellada" (sealed class) de resultados.
- Hay un radar dibujado a mano con CustomPainter y un indicador de "frío o caliente".
- El progreso se guarda en el teléfono (álbum de personajes) a través de un contrato de almacenamiento.
- El modo demo permite "caminar" con botones en el salón de clases: toda la clase puede jugar sin salir.

> 💡 **Dato curioso / ejemplo**
> Pokémon GO tiene el mismo esqueleto: objetivos en el mapa, radar de cercanía, captura al estar a cierta distancia y una colección. La diferencia es que tú vas a entender cómo funciona cada pieza por dentro, y la vas a probar con TDD.

Lo que aprenderás

- Diseñar las reglas de un juego ANTES de programarlo (mini documento de diseño).
- Calcular el "problema directo" de geodesia: dado un punto, un rumbo y una distancia, ¿a dónde llego? (función destino).
- Azar reproducible: inyectar Random con semilla para probar código "aleatorio".
- Distribución uniforme por área y por qué se usa la raíz cuadrada.
- Modelar resultados con sealed classes y switch exhaustivo de Dart 3.
- Radar con CustomPainter y coordenadas polares → cartesianas.
- Persistencia con un contrato y un doble en memoria.
- Probar con playtesting: ajustar radios y dificultad con jugadores reales.

Mapa del proyecto

```
dolores_rally/
  lib/
    dominio/                <- reglas del juego (Dart puro)
      geo.dart              <- distancia, rumbo, ángulos, destino
```

```
    personaje.dart          <- Personaje y Pregunta
    catalogo.dart           <- 6 personajes de la Independencia
    aparicion.dart          <- dónde se esconde cada personaje (azar inyectado)
    radar.dart              <- puntoEnRadar y frío/caliente
    motor_ar.dart           <- ¿qué se ve en pantalla y dónde?
    partida.dart            <- reglas de captura, puntos y huida
    almacen.dart            <- contrato para guardar el progreso
    fuentes.dart  excepciones.dart
  datos/
    fuentes_reales.dart  fuentes_simuladas.dart  almacen_prefs.dart
  presentacion/
    rally_viewmodel.dart  fondo_camara.dart  radar_widget.dart
    pantalla_rally.dart  pantalla_coleccion.dart  pantalla_inicio.dart
  main.dart
test/
  geo_test  aparicion_test  radar_test  motor_ar_test
  partida_test  rally_viewmodel_test  ayudas
```

El ciclo TDD de siempre

🔴 ROJO: escribe la prueba (debe FALLAR)
Escribes la prueba; falla.

🟢 VERDE: el código mínimo para pasar
Código mínimo para que pase.

🔵 REFACTOR: ordena sin romper nada
Ordenas sin romper nada.

⚠️ Cuidado
Los datos históricos del catálogo y las coordenadas son de práctica. Antes de usar el juego con alumnos, verifícalos con fuentes locales (museos, archivo histórico, cronistas de la ciudad). Un juego educativo con un dato erróneo enseña mal; es una excelente oportunidad para colaborar con historiadores locales.

Ronda 0 · Preparar el taller

Paso 0.1 · Proyecto y paquetes

```
$ flutter create dolores_rally
$ cd dolores_rally
$ flutter pub add camera geolocator flutter_compass shared_preferences
$ mkdir lib/dominio lib/datos lib/presentacion
```

📄 **pubspec.yaml**

```
name: dolores_rally
description: Rally de realidad aumentada por Dolores Hidalgo: atrapa personajes de la
Independencia.
publish_to: none
version: 1.0.0

environment:
  sdk: ">=3.0.0 <4.0.0"

# Si "flutter pub get" se queja de una versión, borra la línea y ejecuta:
#   flutter pub add camera geolocator flutter_compass shared_preferences
dependencies:
  flutter:
    sdk: flutter
  camera: ^0.11.0
  geolocator: ^13.0.1
  flutter_compass: ^0.8.1
  shared_preferences: ^2.3.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0

flutter:
  uses-material-design: true
```

Borra test/widget_test.dart y ejecuta flutter pub get. Si alguna versión falla, usa flutter pub add y conserva las que Flutter elija.

Paso 0.2 · Permisos de Android

En android/app/src/main/AndroidManifest.xml, antes de la etiqueta application:

```
<uses-permission android:name="android.permission.CAMERA"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-feature android:name="android.hardware.camera" android:required="false"/>
```

Y en android/app/build.gradle verifica minSdkVersion 21 o superior (el plugin de cámara lo exige).

Paso 0.3 · Permisos de iPhone

```
<key>NSCameraUsageDescription</key>
<string>Usamos la cámara para mostrarte a los personajes escondidos.</string>
<key>NSLocationWhenInUseUsageDescription</key>
```

```
<string>Usamos tu ubicación solo mientras juegas, para saber qué personajes tienes
cerca.</string>
```

> ⚠️ **Cuidado**
> Ubicación "mientras se usa la app", nunca "siempre". Principio de mínimo privilegio: pide solo lo que necesitas.

Paso 0.4 · Verifica el taller

```
$ flutter test
```

Si dice "No tests ran", todo bien. Empezamos por lo más divertido: diseñar el juego en papel.

Ronda 1 · Diseña el juego antes de programarlo

Mini documento de diseño

Los videojuegos profesionales empiezan con un documento de diseño que responde preguntas simples. Responde estas en equipo antes de abrir el editor; después cada respuesta se convertirá en una prueba:

1. ¿Cuál es el objetivo? → Atrapar a los 6 personajes de la Independencia.
2. ¿Cómo se encuentra a un personaje? → Con el radar, el "frío o caliente" y la vista en realidad aumentada.
3. ¿Cuándo se puede atrapar? → Estando a 15 metros o menos.
4. ¿Cómo se atrapa? → Respondiendo bien una pregunta de historia.
5. ¿Qué pasa si fallas? → Tienes 2 intentos; con 2 fallos el personaje huye y reaparece en otro lugar.
6. ¿Cómo se puntúa? → 100 puntos a la primera, 50 a la segunda.
7. ¿Cómo se gana? → Atrapando a los 6. El progreso se guarda en el álbum.
8. ¿Qué tan lejos están? → Entre 60 y 500 metros del Jardín Principal.

La escala de temperatura

Frío o caliente es un juego de niños que funciona en cualquier edad. Definimos los umbrales:

- Ardiente: 15 m o menos (ya puedes atraparlo).
- Caliente: hasta 50 m.
- Tibio: hasta 150 m.
- Frío: hasta 400 m.
- Helado: más lejos.

> 💡 **Dato curioso / ejemplo**
> El diseño de juegos tiene un principio llamado "bucle central": hacer algo → recibir retroalimentación → sentirse recompensado → repetir. Aquí el bucle es: caminar → el letrero se calienta → atrapar → sello en el álbum. Si el bucle es satisfactorio, nadie quiere dejar de jugar.

> ⚠️ **Cuidado**
> ¿Por qué 15 m y no 3 m? El GPS de un teléfono tiene un error de 5 a 20 metros, sobre todo entre edificios. Un radio muy pequeño haría imposible el juego aunque estés parado frente al objetivo. En la Ronda 12 ajustarás este número con jugadores reales.

> 🏆 **RETO PARA TI**
> Agrega una regla nueva al diseño: "pista". Por 20 puntos, el juego dice hacia qué punto cardinal está el personaje más cercano. ¿Qué pruebas escribirías primero? (Pista: usa nombreCardinal y rumboGrados).

Ronda 2 · Geometría de la Tierra (con una función nueva)

El juego necesita cuatro operaciones matemáticas sobre coordenadas GPS. Tres ya las conoces si hiciste la guía Dolores AR (aquí las repasamos), y una es NUEVA.

- distanciaMetros: ¿a cuántos metros está? (fórmula de Haversine).
- rumboGrados: ¿en qué dirección queda? (0° norte, 90° este, 180° sur, 270° oeste).
- diferenciaAngular y suavizarAngulo: ángulos que "dan la vuelta" en 360° (el famoso salto 359° → 1°).
- NUEVA · destino: si camino X metros con rumbo R desde aquí, ¿en qué coordenada termino? La necesitamos para esconder personajes y para que el modo demo permita "caminar".

> 💡 **Dato curioso / ejemplo**
> La función destino resuelve lo que los geodestas llaman "el problema directo": dado un origen, un rumbo y una distancia, hallar el destino. Su inverso (dado origen y destino, hallar rumbo y distancia) es lo que ya teníamos. Son como multiplicar y dividir: operaciones inversas, y por eso podemos probar una con la otra.

Ciclo 2.1 · ROJO: pruebas de destino

Crea test/ayudas.dart y test/geo_test.dart. La prueba estrella es la "ida y vuelta": si caminas 250 m en cualquier dirección y luego mides con distanciaMetros y rumboGrados, debes obtener 250 m y la misma dirección.

📄 **test/ayudas.dart**

```
// Ayudas compartidas por las pruebas (no es una prueba en sí).
import 'package:dolores_rally/dominio/geo.dart';

const origen = Coordenada(21.1561, -100.9318);
```

📄 **test/geo_test.dart**

```
import 'package:dolores_rally/dominio/geo.dart';
import 'package:flutter_test/flutter_test.dart';

import 'ayudas.dart';

void main() {
  group('destino: caminar N metros hacia un rumbo', () {
    test('100 m al norte queda a 100 m con rumbo 0°', () {
      final d = destino(origen, 0, 100);
      expect(distanciaMetros(origen, d), closeTo(100, 0.01));
      expect(rumboGrados(origen, d), closeTo(0, 0.01));
    });

    test('ida y vuelta: distancia y rumbo se conservan en cualquier dirección', () {
      for (final rumbo in [0.0, 45, 90, 135, 180, 225, 270, 315, 359]) {
        final d = destino(origen, rumbo.toDouble(), 250);
        expect(distanciaMetros(origen, d), closeTo(250, 0.05), reason: 'rumbo $rumbo');
        expect(diferenciaAngular(rumboGrados(origen, d), rumbo.toDouble()).abs(),
            lessThan(0.05), reason: 'rumbo $rumbo');
      }
    });

    test('0 metros te deja en el mismo punto', () {
      final d = destino(origen, 123, 0);
      expect(distanciaMetros(origen, d), closeTo(0, 1e-6));
```

```
    });
  });

  group('recordatorio de la guía anterior', () {
    test('un grado de latitud mide unos 111.2 km', () {
      expect(distanciaMetros(const Coordenada(0, 0), const Coordenada(1, 0)), closeTo(111195,
5));
    });

    test('diferenciaAngular cruza el norte por el camino corto', () {
      expect(diferenciaAngular(10, 350), 20);
      expect(diferenciaAngular(350, 10), -20);
    });

    test('suavizarAngulo no se rompe en el salto 350° → 10°', () {
      expect(suavizarAngulo(350, 10, alfa: 0.5), closeTo(0, 1e-9));
    });

    test('posicionHorizontal: centro, borde y fuera de campo', () {
      expect(posicionHorizontal(0), 0.5);
      expect(posicionHorizontal(30), closeTo(1.0, 1e-9));
      expect(posicionHorizontal(31), isNull);
    });
  });
}
```

🔴 **ROJO: escribe la prueba (debe FALLAR)**

flutter test falla: geo.dart no existe. Observa que las pruebas del recordatorio (distancia, ángulos, campo de visión) también están aquí; sirven como red de seguridad.

Ciclo 2.2 · VERDE: geo.dart completo

📄 **lib/dominio/geo.dart**

```
import 'dart:math' as math;

/// Matemáticas de la realidad aumentada por ubicación.
/// Dart puro: sin Flutter, sin GPS, sin cámara. Se prueba en milisegundos.

class Coordenada {
  final double lat;
  final double lon;
  const Coordenada(this.lat, this.lon);

  @override
  String toString() => '(${lat.toStringAsFixed(5)}, ${lon.toStringAsFixed(5)})';
}

const double _radioTierra = 6371000.0; // metros
double _rad(double grados) => grados * math.pi / 180;
double _grados(double rad) => rad * 180 / math.pi;

/// Distancia en metros entre dos puntos de la Tierra (fórmula de Haversine).
double distanciaMetros(Coordenada a, Coordenada b) {
  final dLat = _rad(b.lat - a.lat);
  final dLon = _rad(b.lon - a.lon);
  final h = math.pow(math.sin(dLat / 2), 2) +
      math.cos(_rad(a.lat)) * math.cos(_rad(b.lat)) * math.pow(math.sin(dLon / 2), 2);
  return 2 * _radioTierra * math.asin(math.sqrt(h.clamp(0.0, 1.0).toDouble()));
}
```

```
/// Rumbo (0° = norte, 90° = este, 180° = sur, 270° = oeste) para ir de [a] a [b].
double rumboGrados(Coordenada a, Coordenada b) {
  final dLon = _rad(b.lon - a.lon);
  final lat1 = _rad(a.lat);
  final lat2 = _rad(b.lat);
  final y = math.sin(dLon) * math.cos(lat2);
  final x = math.cos(lat1) * math.sin(lat2) - math.sin(lat1) * math.cos(lat2) *
math.cos(dLon);
  return normalizar360(_grados(math.atan2(y, x)));
}

/// Lleva cualquier ángulo al rango 0 <= g < 360.
double normalizar360(double g) => ((g % 360) + 360) % 360;

/// Cuánto hay que girar (en grados, de -180 a 180) para pasar de [actual] a [objetivo].
/// Positivo = girar a la derecha. Resuelve el problema del salto 359° -> 1°.
double diferenciaAngular(double objetivo, double actual) {
  var d = normalizar360(objetivo - actual);
  if (d > 180) d -= 360;
  return d;
}

/// Filtro pasa-bajas circular: suaviza la brújula sin romperse en el salto 359°/0°.
/// [alfa] cerca de 0 = muy suave y lento; cerca de 1 = rápido y tembloroso.
double suavizarAngulo(double anterior, double nuevo, {double alfa = 0.2}) =>
    normalizar360(anterior + alfa * diferenciaAngular(nuevo, anterior));

/// Posición horizontal en pantalla (0 = borde izquierdo, 1 = borde derecho)
/// o null si el objeto queda fuera del campo de visión de la cámara.
double? posicionHorizontal(double diferencia, {double campoVision = 60}) {
  if (diferencia.abs() > campoVision / 2) return null;
  return 0.5 + diferencia / campoVision;
}

String formatearDistancia(double metros) =>
    metros < 1000 ? '${metros.round()} m' : '${(metros / 1000).toStringAsFixed(1)} km';

String nombreCardinal(double grados) {
  const puntos = ['N', 'NE', 'E', 'SE', 'S', 'SO', 'O', 'NO'];
  return puntos[((normalizar360(grados) + 22.5) ~/ 45) % 8];
}

/// Punto al que llegas si caminas [metros] desde [origen] con rumbo [rumbo].
/// Es la operación inversa de distancia + rumbo (problema "directo" de geodesia).
Coordenada destino(Coordenada origen, double rumbo, double metros) {
  final d = metros / _radioTierra; // distancia angular
  final br = _rad(rumbo);
  final lat1 = _rad(origen.lat);
  final lon1 = _rad(origen.lon);
  final lat2 = math.asin(math.sin(lat1) * math.cos(d) + math.cos(lat1) * math.sin(d) *
math.cos(br));
  final lon2 = lon1 +
      math.atan2(math.sin(br) * math.sin(d) * math.cos(lat1),
          math.cos(d) - math.sin(lat1) * math.sin(lat2));
  return Coordenada(_grados(lat2), ((_grados(lon2) + 540) % 360) - 180);
}
```

Lectura guiada de destino

9. metros / radioTierra convierte la distancia en un "ángulo de viaje" sobre la esfera (distancia angular).

10. La latitud nueva sale de combinar la latitud inicial con ese ángulo de viaje y el rumbo (ley de cosenos esférica).
11. La longitud nueva usa atan2 para que el signo y el cuadrante salgan bien sin trucos.
12. El último paso ((lon + 540) % 360) − 180 mantiene la longitud entre −180° y 180° aunque cruces el meridiano opuesto.

🔵 REFACTOR: ordena sin romper nada

Ejecuta flutter test. Las pruebas de ida y vuelta recorren 9 direcciones: si cambias una fórmula y rompes SOLO una dirección (error muy común), la prueba te dice cuál.

💡 Dato curioso / ejemplo

Una sola prueba de "100 m al norte" habría pasado aunque la fórmula tuviera un error en el este. Por eso probamos varias direcciones: los errores matemáticos suelen esconderse en un cuadrante.

🏅 INSIGNIA DESBLOQUEADA: Navegante de coordenadas

Ronda 3 · Los personajes y sus preguntas

Modelo de datos

Una Pregunta tiene el texto, las opciones y el índice de la respuesta correcta. Un Personaje junta nombre, emoji, biografía corta y su pregunta. Son datos inmutables (const).

📄 **lib/dominio/personaje.dart**

```
class Pregunta {
  final String texto;
  final List<String> opciones;
  final int correcta; // índice de la opción correcta

  const Pregunta(this.texto, this.opciones, this.correcta);

  bool esCorrecta(int opcion) => opcion == correcta;
}

class Personaje {
  final String id;
  final String nombre;
  final String emoji;
  final String bio;
  final Pregunta pregunta;

  const Personaje({
    required this.id,
    required this.nombre,
    required this.emoji,
    required this.bio,
    required this.pregunta,
  });
}
```

El catálogo

📄 **lib/dominio/catalogo.dart**

```
import 'geo.dart';
import 'personaje.dart';

/// Centro del juego en Dolores Hidalgo (Jardín Principal, aproximado).
const plazaPrincipal = Coordenada(21.15610, -100.93180);

/// ⚠️ Verifica estos datos con fuentes históricas locales antes de usarlos en clase.
const personajes = <Personaje>[
  Personaje(
    id: 'hidalgo',
    nombre: 'Miguel Hidalgo',
    emoji: '🔔',
    bio: 'Cura de Dolores. Convocó al pueblo a levantarse en armas con el Grito de Dolores.',
    pregunta: Pregunta(
      '¿En qué año dio Hidalgo el Grito de Dolores?',
      ['1810', '1821', '1857', '1910'],
      0,
    ),
  ),
  Personaje(
    id: 'allende',
    nombre: 'Ignacio Allende',
```

```
    emoji: '⚔️',
    bio: 'Militar criollo y uno de los principales jefes del movimiento insurgente.',
    pregunta: Pregunta(
      '¿Con qué rango militar participó Ignacio Allende?',
      ['Almirante', 'Capitán', 'Cardenal', 'Virrey'],
      1,
    ),
  ),
  Personaje(
    id: 'josefa',
    nombre: 'Josefa Ortiz de Domínguez',
    emoji: '🗝️',
    bio: 'Participó en la conspiración de Querétaro y ayudó a avisar a los insurgentes.',
    pregunta: Pregunta(
      '¿Con qué apodo se conoce a Josefa Ortiz de Domínguez?',
      ['La Malinche', 'La Güera', 'La Corregidora', 'La Adelita'],
      2,
    ),
  ),
  Personaje(
    id: 'aldama',
    nombre: 'Juan Aldama',
    emoji: '🐎',
    bio: 'Capitán insurgente que llegó a Dolores con noticias urgentes sobre la
conspiración.',
    pregunta: Pregunta(
      '¿Qué noticia llevó Juan Aldama a Hidalgo en Dolores?',
      [
        'Que la conspiración había sido descubierta',
        'Que había llegado el nuevo virrey',
        'Que la guerra había terminado',
        'Que se cancelaba la fiesta del pueblo',
      ],
      0,
    ),
  ),
  Personaje(
    id: 'abasolo',
    nombre: 'Mariano Abasolo',
    emoji: '🛡️',
    bio: 'Capitán insurgente nacido en Dolores, participó desde el inicio del movimiento.',
    pregunta: Pregunta(
      '¿En qué ciudad nació el capitán Mariano Abasolo?',
      ['Monterrey', 'Mérida', 'Oaxaca', 'Dolores Hidalgo'],
      3,
    ),
  ),
  Personaje(
    id: 'leona',
    nombre: 'Leona Vicario',
    emoji: '📰',
    bio: 'Apoyó a los insurgentes con dinero, mensajes e información, y es recordada como
periodista.',
    pregunta: Pregunta(
      '¿Qué actividad destacó a Leona Vicario además de apoyar la insurgencia?',
      ['Periodista', 'Marinera', 'Cantante de ópera', 'Arquitecta'],
      0,
    ),
  ),
];
```

💡 Dato curioso / ejemplo

La respuesta correcta no siempre está en la primera opción: se reparte (0, 1, 2, 3...). Si la correcta siempre fuera la A, los jugadores lo notarían en dos minutos. Con tu propio equipo, revisen que el orden esté repartido.

⚠️ **Cuidado**

Preguntas de historia = responsabilidad. Cada pregunta debe tener UNA respuesta indiscutible y verificada. Evita preguntas con matices o fechas con versiones distintas. Pide a un docente de historia que revise el catálogo.

🏆 **RETO PARA TI**

Agrega dos personajes más (por ejemplo, de la región de Guanajuato) con su biografía y pregunta. ¿Se rompe alguna prueba? ¿Por qué no? (Pista: pregunta qué asumen las pruebas sobre el número de personajes).

🏅 **INSIGNIA DESBLOQUEADA: Historiador del juego**

Ronda 4 · Esconder personajes: azar que se puede probar

El problema

"Aleatorio" y "probable con TDD" parecen enemigos: si el resultado cambia en cada ejecución, ¿cómo escribo una prueba con un valor esperado? Hay dos respuestas, y usaremos ambas:

13. Inyectar el generador: en lugar de crear Random() dentro de la función, lo recibimos como parámetro. En el juego pasamos Random() (sin semilla: distinto cada vez). En las pruebas pasamos Random(42): con la misma semilla produce SIEMPRE la misma secuencia.
14. Probar propiedades, no valores: no podemos saber dónde caerá cada punto, pero sí que TODOS estén entre 60 y 500 metros y que se repartan por todas las direcciones.

> 💡 **Dato curioso / ejemplo**
> Una semilla es como la receta de un pastel: el generador sigue los mismos pasos, así que siempre sale el mismo pastel. Random(42) es una receta conocida; Random() usa una receta distinta cada vez (basada en la hora).

El truco de la raíz cuadrada

Imagina que escoges la distancia al azar entre 60 y 500 m y la dirección al azar. ¿Quedan los puntos repartidos parejo? NO: se amontonan cerca del centro, porque un círculo pequeño tiene mucha menos área que uno grande. Piénsalo como lanzar dardos a una diana: si eliges primero el radio al azar, los dardos se amontonan en el centro.

Para repartir parejo por ÁREA, la distancia es: $d = \sqrt{(rMin^2 + u \cdot (rMax^2 - rMin^2))}$, donde u es un número al azar entre 0 y 1. La raíz cuadrada "compensa" que las coronas lejanas tienen más área.

> 💡 **Dato curioso / ejemplo**
> Esta técnica se llama muestreo uniforme sobre un anillo. La usan los videojuegos para repartir enemigos, las simulaciones para esparcir partículas y hasta los robots que dispersan semillas. Un truco de matemáticas de preparatoria con uso muy profesional.

Ciclo 4.1 · ROJO

📄 **test/aparicion_test.dart**

```
import 'dart:math';

import 'package:dolores_rally/dominio/aparicion.dart';
import 'package:dolores_rally/dominio/catalogo.dart';
import 'package:dolores_rally/dominio/geo.dart';
import 'package:flutter_test/flutter_test.dart';

import 'ayudas.dart';

void main() {
  test('genera una aparición por personaje', () {
    final a = generarApariciones(centro: origen, personajes: personajes, random: Random(1));
    expect(a.length, personajes.length);
    expect(a.map((x) => x.personaje.id).toSet().length, personajes.length);
  });
```

```
  test('todas quedan entre el radio mínimo y el máximo', () {
    final a = generarApariciones(
        centro: origen, personajes: personajes, random: Random(7), radioMin: 60, radioMax:
500);
    for (final x in a) {
      final d = distanciaMetros(origen, x.coordenada);
      expect(d, inInclusiveRange(59.9, 500.1));
    }
  });

  test('con la misma semilla el "azar" se repite (pruebas confiables)', () {
    final a = generarApariciones(centro: origen, personajes: personajes, random: Random(42));
    final b = generarApariciones(centro: origen, personajes: personajes, random: Random(42));
    for (var i = 0; i < a.length; i++) {
      expect(a[i].coordenada.lat, b[i].coordenada.lat);
      expect(a[i].coordenada.lon, b[i].coordenada.lon);
    }
  });

  test('con semillas distintas los lugares son distintos', () {
    final a = generarApariciones(centro: origen, personajes: personajes, random: Random(1));
    final b = generarApariciones(centro: origen, personajes: personajes, random: Random(2));
    expect(a.first.coordenada.lat, isNot(b.first.coordenada.lat));
  });

  test('los puntos se reparten por TODAS las direcciones', () {
    final r = Random(5);
    final rumbos = [
      for (var i = 0; i < 400; i++) rumboGrados(origen, puntoAleatorio(origen, 60, 500, r))
    ];
    for (final cuadrante in [0, 90, 180, 270]) {
      expect(rumbos.where((g) => g >= cuadrante && g < cuadrante + 90).length,
greaterThan(60),
          reason: 'cuadrante $cuadrante');
    }
  });
}
```

Observa la última prueba: genera 400 puntos y exige que cada cuadrante (norte-este, sur-este, sur-oeste, norte-oeste) tenga más de 60. Es una prueba estadística: si el azar estuviera sesgado hacia un lado, falla.

Ciclo 4.2 · VERDE

📄 **lib/dominio/aparicion.dart**

```
import 'dart:math' as math;

import 'geo.dart';
import 'personaje.dart';

/// Un personaje escondido en un punto del mapa.
class Aparicion {
  final Personaje personaje;
  final Coordenada coordenada;
  const Aparicion(this.personaje, this.coordenada);
}

/// Punto al azar entre [min] y [max] metros del [centro].
/// Se usa raíz cuadrada para que los puntos queden repartidos de forma PAREJA
/// por el área (sin ella, se amontonarían cerca del centro).
Coordenada puntoAleatorio(Coordenada centro, double min, double max, math.Random random) {
  final distancia = math.sqrt(min * min + random.nextDouble() * (max * max - min * min));
```

```
  return destino(centro, random.nextDouble() * 360, distancia);
}

/// El [random] se inyecta: con una semilla fija (Random(42)) el "azar" es repetible
/// y las pruebas son confiables; en el juego real se usa Random() sin semilla.
List<Aparicion> generarApariciones({
  required Coordenada centro,
  required List<Personaje> personajes,
  required math.Random random,
  double radioMin = 60,
  double radioMax = 500,
}) =>
    [for (final p in personajes) Aparicion(p, puntoAleatorio(centro, radioMin, radioMax,
random))];
```

🔵 REFACTOR: ordena sin romper nada

Observa que puntoAleatorio está separada de generarApariciones: la usaremos otra vez para REUBICAR a un personaje que se escapa. Una función pequeña y reutilizable vale más que copiar y pegar.

🏆 RETO PARA TI

Agrega una regla: dos personajes no pueden aparecer a menos de 30 metros entre sí. Escribe primero la prueba. (Pista: ¿qué pasa si la regla nunca puede cumplirse? Piensa en un límite de intentos).

🏅 INSIGNIA DESBLOQUEADA: Maestro del azar

Ronda 5 · Radar y "frío o caliente"

Del mapa al radar: coordenadas polares

El radar muestra al jugador en el centro, "arriba" es hacia donde mira, y cada personaje es un punto. Para ubicarlo necesitamos dos datos que ya sabemos calcular:

- Distancia d → qué tan lejos del centro del radar (fracción f = d / alcance).
- Ángulo relativo a (la diferencia entre el rumbo al personaje y hacia donde miras) → en qué dirección.

Eso es una coordenada polar (f, a). La pantalla usa coordenadas cartesianas (x, y). La conversión es de trigonometría de preparatoria: x = f · sen(a) e y = −f · cos(a). El signo menos en y es porque en las pantallas "arriba" es la dirección negativa del eje y.

```
       arriba = hacia donde miras
             y = -1
              |
  x = -1 ---- YO ---- x = +1
              |
             y = +1
       abajo = a tus espaldas
```

> 💡 **Dato curioso / ejemplo**
> Los radares de verdad (de barcos, aviones y videojuegos) usan exactamente esta matemática. Cuando un submarino aparece "a las 3 en punto, a 2 km", es un ángulo y una distancia convertidos a x, y.

Ciclo 5.1 · ROJO

📄 **test/radar_test.dart**

```dart
import 'package:dolores_rally/dominio/geo.dart';
import 'package:dolores_rally/dominio/radar.dart';
import 'package:flutter_test/flutter_test.dart';

import 'ayudas.dart';

void main() {
  group('puntoEnRadar', () {
    final norte = destino(origen, 0, 250);
    final este = destino(origen, 90, 250);
    final sur = destino(origen, 180, 250);

    test('algo al norte mirando al norte aparece ARRIBA a media distancia', () {
      final (x, y) = puntoEnRadar(usuario: origen, rumbo: 0, objetivo: norte, alcance: 500)!;
      expect(x, closeTo(0, 0.01));
      expect(y, closeTo(-0.5, 0.01));
    });

    test('algo al este mirando al norte aparece a la DERECHA', () {
      final (x, y) = puntoEnRadar(usuario: origen, rumbo: 0, objetivo: este, alcance: 500)!;
      expect(x, closeTo(0.5, 0.01));
      expect(y, closeTo(0, 0.01));
    });

    test('al girar hacia el este, ese mismo objeto pasa ARRIBA', () {
      final (x, y) = puntoEnRadar(usuario: origen, rumbo: 90, objetivo: este, alcance: 500)!;
      expect(x, closeTo(0, 0.01));
      expect(y, closeTo(-0.5, 0.01));
```

```
    });

    test('algo al sur mirando al norte aparece ABAJO', () {
      final (_, y) = puntoEnRadar(usuario: origen, rumbo: 0, objetivo: sur, alcance: 500)!;
      expect(y, closeTo(0.5, 0.01));
    });

    test('más lejos que el alcance no aparece', () {
      expect(puntoEnRadar(usuario: origen, rumbo: 0, objetivo: destino(origen, 0, 800),
alcance: 500), isNull);
    });
  });

  group('temperaturaPorDistancia (frío o caliente)', () {
    test('umbrales', () {
      expect(temperaturaPorDistancia(10), Temperatura.ardiente);
      expect(temperaturaPorDistancia(15), Temperatura.ardiente);
      expect(temperaturaPorDistancia(40), Temperatura.caliente);
      expect(temperaturaPorDistancia(100), Temperatura.tibio);
      expect(temperaturaPorDistancia(300), Temperatura.frio);
      expect(temperaturaPorDistancia(900), Temperatura.helado);
    });

    test('mientras más lejos, nunca se pone más caliente', () {
      var anterior = Temperatura.ardiente.index;
      for (var m = 0.0; m <= 1000; m += 5) {
        final actual = temperaturaPorDistancia(m).index;
        expect(actual, greaterThanOrEqualTo(anterior));
        anterior = actual;
      }
    });
  });
}
```

Mira el cuarteto de pruebas de orientación: algo al norte mirando al norte va ARRIBA; algo al este mirando al norte va a la DERECHA; al girar hacia el este ese objeto pasa ARRIBA; algo al sur va ABAJO. Si tu radar pasa esas cuatro, sabes que está bien orientado.

Ciclo 5.2 · VERDE

📄 **lib/dominio/radar.dart**

```
import 'dart:math' as math;

import 'geo.dart';

/// Posición de [objetivo] en un radar circular centrado en el jugador.
/// Devuelve (x, y) entre -1 y 1. Arriba (y negativo) = hacia donde miras.
/// Null si queda más lejos que [alcance] metros.
(double, double)? puntoEnRadar({
  required Coordenada usuario,
  required double rumbo,
  required Coordenada objetivo,
  double alcance = 500,
}) {
  final d = distanciaMetros(usuario, objetivo);
  if (d > alcance) return null;
  final dif = diferenciaAngular(rumboGrados(usuario, objetivo), rumbo) * math.pi / 180;
  final f = d / alcance;
  return (f * math.sin(dif), -f * math.cos(dif));
}
```

```
enum Temperatura { helado, frio, tibio, caliente, ardiente }

/// El juego de "frío o caliente": qué tan cerca estás de tu objetivo.
Temperatura temperaturaPorDistancia(double metros) {
  if (metros <= 15) return Temperatura.ardiente;
  if (metros <= 50) return Temperatura.caliente;
  if (metros <= 150) return Temperatura.tibio;
  if (metros <= 400) return Temperatura.frio;
  return Temperatura.helado;
}

extension TextoTemperatura on Temperatura {
  String get texto => switch (this) {
        Temperatura.helado => '🥶 Helado',
        Temperatura.frio => '❄️ Frío',
        Temperatura.tibio => '🙂 Tibio',
        Temperatura.caliente => '🔥 Caliente',
        Temperatura.ardiente => '🌋 ¡Ardiendo! Atrápalo',
      };
}
```

💡 Dato curioso / ejemplo

Fíjate en el valor devuelto: (double, double)? . Es un "record" nulo de Dart 3: un par de números que puede no existir (null = fuera de alcance). El compilador te obliga a revisar el null antes de dibujar.

⚠️ Cuidado

La prueba "mientras más lejos, nunca se pone más caliente" recorre distancias de 0 a 1000 m de 5 en 5. Es una prueba de propiedad: no importa dónde pongas los umbrales, la temperatura nunca debe enfriarse al acercarte ni calentarse al alejarte.

🏅 INSIGNIA DESBLOQUEADA: Operador de radar

Ronda 6 · El motor de realidad aumentada

Ya sabemos dónde están los personajes y hacia dónde mira el jugador. Falta responder: ¿cuáles caen dentro de lo que ve la cámara y en qué parte de la pantalla? La cámara de un teléfono ve un "abanico" de unos 60° (campo de visión). Un personaje justo al frente va en el centro; uno en el borde del abanico va en el borde de la pantalla; uno fuera del abanico no se dibuja.

```
x = 0.5 + diferenciaAngular / campoDeVision

 -30°            0°            +30°
  x = 0        x = 0.5         x = 1
(izquierda)    (centro)       (derecha)
```

Ciclo 6.1 · ROJO

📄 **test/motor_ar_test.dart**

```
import 'package:dolores_rally/dominio/aparicion.dart';
import 'package:dolores_rally/dominio/geo.dart';
import 'package:dolores_rally/dominio/motor_ar.dart';
import 'package:dolores_rally/dominio/personaje.dart';
import 'package:flutter_test/flutter_test.dart';

import 'ayudas.dart';

Aparicion en(String id, double rumbo, double metros) => Aparicion(
      Personaje(
        id: id, nombre: id, emoji: '🎯', bio: '',
        pregunta: const Pregunta('?', ['a', 'b'], 0),
      ),
      destino(origen, rumbo, metros),
    );

void main() {
  test('un personaje justo adelante aparece en el centro de la pantalla', () {
    final v = calcularVisibles(usuario: origen, rumbo: 0, apariciones: [en('a', 0, 100)]);
    expect(v.single.x, closeTo(0.5, 0.01));
    expect(v.single.distancia, closeTo(100, 0.5));
  });

  test('uno a 90° de donde miras no aparece; al girar hacia él, sí', () {
    final a = en('a', 90, 100);
    expect(calcularVisibles(usuario: origen, rumbo: 0, apariciones: [a]), isEmpty);
    expect(calcularVisibles(usuario: origen, rumbo: 90, apariciones: [a]), hasLength(1));
  });

  test('funciona al cruzar el norte (miras a 350° y está a 5°)', () {
    final v = calcularVisibles(usuario: origen, rumbo: 350, apariciones: [en('a', 5, 100)]);
    expect(v, hasLength(1));
    expect(v.single.x, greaterThan(0.5));
  });

  test('los lejanos van primero y se ven más pequeños', () {
    final v = calcularVisibles(
        usuario: origen, rumbo: 0, apariciones: [en('cerca', 0, 40), en('lejos', 0, 450)]);
    expect(v.map((x) => x.aparicion.personaje.id), ['lejos', 'cerca']);
    expect(v.last.escala, greaterThan(v.first.escala));
  });
```

```
  test('más allá del radio máximo se descartan', () {
    final v = calcularVisibles(
        usuario: origen, rumbo: 0, apariciones: [en('a', 0, 900)], radioMax: 600);
    expect(v, isEmpty);
  });
}
```

Ciclo 6.2 · VERDE

📄 **lib/dominio/motor_ar.dart**

```
import 'aparicion.dart';
import 'geo.dart';

class Visible {
  final Aparicion aparicion;
  final double distancia;
  final double x; // 0 izquierda .. 1 derecha
  final double profundidad; // 0 cerca .. 1 lejos
  final double escala; // 1.2 cerca .. 0.6 lejos
  const Visible(this.aparicion, this.distancia, this.x, this.profundidad, this.escala);
}

/// ¿Qué personajes caen dentro de lo que ve la cámara?
List<Visible> calcularVisibles({
  required Coordenada usuario,
  required double rumbo,
  required Iterable<Aparicion> apariciones,
  double campoVision = 60,
  double radioMax = 600,
}) {
  final res = <Visible>[];
  for (final a in apariciones) {
    final d = distanciaMetros(usuario, a.coordenada);
    if (d > radioMax) continue;
    final dif = diferenciaAngular(rumboGrados(usuario, a.coordenada), rumbo);
    final x = posicionHorizontal(dif, campoVision: campoVision);
    if (x == null) continue;
    final prof = (d / radioMax).clamp(0.0, 1.0).toDouble();
    res.add(Visible(a, d, x, prof, 1.2 - 0.6 * prof));
  }
  res.sort((a, b) => b.distancia.compareTo(a.distancia));
  return res;
}
```

Tres trucos de profundidad

- Escala: lo cercano se ve hasta 1.2 veces; lo lejano se encoge hasta 0.6.
- Profundidad: lo lejano "sube" hacia el horizonte en la pantalla.
- Orden: se ordenan de lejos a cerca para que lo cercano tape a lo lejano.

> ⚠️ **Cuidado**
> La prueba "funciona al cruzar el norte" (miras a 350° y el personaje está a 5°) protege contra el bug clásico de las apps con brújula: todo funciona excepto al mirar al norte.

🏅 **INSIGNIA DESBLOQUEADA: Ingeniero de RA**

Ronda 7 · Las reglas del juego (la clase Partida)

Modelar resultados con sealed class

Atrapar a un personaje puede terminar de TRES formas distintas, cada una con datos distintos: Capturado (con puntos), Fallo (con intentos restantes) o SeEscapo (con nueva posición). En lugar de devolver un número o un texto, modelamos cada resultado como una clase. Con "sealed class" Dart sabe que SOLO existen esas tres, y el switch de la pantalla te obliga a cubrirlas todas: si agregas un cuarto resultado, el compilador marca error en todos los switch que lo olviden.

> 💡 **Dato curioso / ejemplo**
> Es como el semáforo: solo hay verde, amarillo y rojo. Si un conductor (switch) no sabe qué hacer con el amarillo, el compilador no lo deja salir del garaje.

Ciclo 7.1 · ROJO: diez pruebas, diez reglas

📄 **test/partida_test.dart**

```
import 'package:dolores_rally/dominio/aparicion.dart';
import 'package:dolores_rally/dominio/excepciones.dart';
import 'package:dolores_rally/dominio/geo.dart';
import 'package:dolores_rally/dominio/partida.dart';
import 'package:dolores_rally/dominio/personaje.dart';
import 'package:flutter_test/flutter_test.dart';

import 'ayudas.dart';

const _p = Personaje(
  id: 'x',
  nombre: 'Personaje X',
  emoji: '🎯',
  bio: '',
  pregunta: Pregunta('¿2+2?', ['3', '4', '5'], 1),
);

const _escondite = Coordenada(21.2, -100.9);

Partida crear({List<Coordenada>? reubicaciones}) {
  final cola = [...?reubicaciones];
  return Partida(
    [Aparicion(_p, origen)],
    reubicar: () => cola.isEmpty ? _escondite : cola.removeAt(0),
  );
}

void main() {
  test('respuesta correcta y cerca: captura con 100 puntos', () {
    final partida = crear();
    final r = partida.capturar('x', 1, 10);
    expect(r, isA<Capturado>());
    expect((r as Capturado).puntos, 100);
    expect(partida.puntos, 100);
    expect(partida.estaCapturado('x'), true);
    expect(partida.terminada, true);
  });

  test('si estás lejos, ni siquiera se puede intentar', () {
    final partida = crear();
```

```
    expect(() => partida.capturar('x', 1, 80), throwsA(isA<LejosException>()));
    expect(partida.puntos, 0);
  });

  test('a exactamente 15 m todavía cuenta', () {
    expect(crear().capturar('x', 1, 15), isA<Capturado>());
  });

  test('una respuesta incorrecta da otra oportunidad', () {
    final partida = crear();
    final r = partida.capturar('x', 0, 5);
    expect(r, isA<Fallo>());
    expect((r as Fallo).intentosRestantes, 1);
    expect(partida.estaCapturado('x'), false);
  });

  test('acertar en el segundo intento vale solo 50 puntos', () {
    final partida = crear();
    partida.capturar('x', 0, 5);
    final r = partida.capturar('x', 1, 5) as Capturado;
    expect(r.puntos, 50);
  });

  test('dos fallos: el personaje se escapa y reaparece en otro lugar', () {
    final partida = crear(reubicaciones: [const Coordenada(21.3, -100.8)]);
    partida.capturar('x', 0, 5);
    final r = partida.capturar('x', 2, 5);
    expect(r, isA<SeEscapo>());
    final nueva = (r as SeEscapo).nuevaPosicion;
    expect(nueva.lat, 21.3);
    expect(partida.pendientes.single.coordenada.lat, 21.3);
  });

  test('tras escaparse, el contador de fallos se reinicia', () {
    final partida = crear();
    partida.capturar('x', 0, 5);
    partida.capturar('x', 0, 5); // se escapa
    final r = partida.capturar('x', 0, 5); // primer fallo del nuevo ciclo
    expect(r, isA<Fallo>());
  });

  test('no se puede capturar dos veces al mismo personaje', () {
    final partida = crear();
    partida.capturar('x', 1, 5);
    expect(() => partida.capturar('x', 1, 5), throwsA(isA<YaCapturadoException>()));
    expect(partida.puntos, 100);
  });

  test('un personaje que no existe lanza error con nombre propio', () {
    expect(() => crear().capturar('nadie', 0, 5),
throwsA(isA<PersonajeDesconocidoException>()));
  });

  test('restaurar marca como capturados a los guardados e ignora ids raros', () {
    final partida = crear();
    partida.restaurar(['x', 'fantasma']);
    expect(partida.capturados, {'x'});
    expect(partida.pendientes, isEmpty);
  });
}
```

Lee los nombres de las pruebas como si fueran el reglamento del juego. Casos de frontera importantes: a EXACTAMENTE 15 m todavía cuenta; tras escaparse, el contador de fallos se reinicia; capturar dos veces lanza error.

Ciclo 7.2 · VERDE

📄 **lib/dominio/excepciones.dart**

```
abstract class ErrorDeApp implements Exception {
  const ErrorDeApp();
}

class PermisoUbicacionException extends ErrorDeApp {
  const PermisoUbicacionException();
  @override
  String toString() =>
      'Necesito el permiso de ubicación para jugar. Actívalo en Ajustes.';
}

class UbicacionApagadaException extends ErrorDeApp {
  const UbicacionApagadaException();
  @override
  String toString() => 'El GPS está apagado. Actívalo para jugar.';
}

class SinBrujulaException extends ErrorDeApp {
  const SinBrujulaException();
  @override
  String toString() => 'Este dispositivo no tiene brújula. Usa el modo demo.';
}

class LejosException extends ErrorDeApp {
  final double metros;
  const LejosException(this.metros);
  @override
  String toString() =>
      'Estás a ${metros.round()} m. Acércate a menos de 15 m para atraparlo.';
}

class YaCapturadoException extends ErrorDeApp {
  const YaCapturadoException();
  @override
  String toString() => 'Ya atrapaste a este personaje.';
}

class PersonajeDesconocidoException extends ErrorDeApp {
  const PersonajeDesconocidoException();
  @override
  String toString() => 'Ese personaje no existe en el juego.';
}

/// Mensaje que SÍ puede ver el jugador (sin detalles internos).
String mensajeSeguro(Object error) =>
    error is ErrorDeApp ? error.toString() : 'Ocurrió un error inesperado. Intenta de nuevo.';
```

📄 **lib/dominio/partida.dart**

```
import 'aparicion.dart';
import 'excepciones.dart';
import 'geo.dart';
```

```
sealed class ResultadoCaptura {
  const ResultadoCaptura();
}

class Capturado extends ResultadoCaptura {
  final int puntos;
  const Capturado(this.puntos);
}

class Fallo extends ResultadoCaptura {
  final int intentosRestantes;
  const Fallo(this.intentosRestantes);
}

/// Fallaste demasiado: el personaje huye y reaparece en otro lugar.
class SeEscapo extends ResultadoCaptura {
  final Coordenada nuevaPosicion;
  const SeEscapo(this.nuevaPosicion);
}

/// Reglas del juego. Dart puro: no sabe nada de GPS, cámara ni pantallas.
class Partida {
  static const double radioCaptura = 15;
  static const int maxFallos = 2;
  static const int puntosPrimerIntento = 100;
  static const int puntosSegundoIntento = 50;

  final Map<String, Aparicion> _apariciones;
  final Coordenada Function() _reubicar;
  final Set<String> _capturados = {};
  final Map<String, int> _fallos = {};
  int puntos = 0;

  Partida(List<Aparicion> apariciones, {required Coordenada Function() reubicar})
      : _apariciones = {for (final a in apariciones) a.personaje.id: a},
        _reubicar = reubicar;

  List<Aparicion> get todas => _apariciones.values.toList();
  List<Aparicion> get pendientes =>
      _apariciones.values.where((a) => !_capturados.contains(a.personaje.id)).toList();
  Set<String> get capturados => Set.unmodifiable(_capturados);
  bool get terminada => _capturados.length == _apariciones.length;
  bool estaCapturado(String id) => _capturados.contains(id);

  /// Marca como capturados a los que el jugador ya tenía de partidas anteriores.
  void restaurar(Iterable<String> ids) {
    _capturados.addAll(ids.where(_apariciones.containsKey));
  }

  ResultadoCaptura capturar(String id, int opcion, double distanciaMetros) {
    final a = _apariciones[id];
    if (a == null) throw const PersonajeDesconocidoException();
    if (_capturados.contains(id)) throw const YaCapturadoException();
    if (distanciaMetros > radioCaptura) throw LejosException(distanciaMetros);

    final fallosPrevios = _fallos[id] ?? 0;
    if (a.personaje.pregunta.esCorrecta(opcion)) {
      final ganados = fallosPrevios == 0 ? puntosPrimerIntento : puntosSegundoIntento;
      puntos += ganados;
      _capturados.add(id);
      return Capturado(ganados);
    }
```

```
    final fallos = fallosPrevios + 1;
    if (fallos >= maxFallos) {
      _fallos[id] = 0;
      final nueva = _reubicar();
      _apariciones[id] = Aparicion(a.personaje, nueva);
      return SeEscapo(nueva);
    }
    _fallos[id] = fallos;
    return Fallo(maxFallos - fallos);
  }
}
```

Lectura guiada de capturar()

15. Primero las validaciones que lanzan excepción: personaje desconocido, ya capturado, demasiado lejos. Orden importante: si estás lejos, ni siquiera se evalúa la respuesta.
16. Si la respuesta es correcta, los puntos dependen de cuántos fallos llevaba (100 o 50).
17. Si es incorrecta, se suma un fallo. Al llegar a 2, el personaje huye: se llama a reubicar() y se reinician los fallos.
18. reubicar es una función recibida por el constructor (inyección de dependencias): la Partida no sabe cómo se elige el lugar nuevo. En las pruebas devuelve un punto fijo; en el juego, uno al azar.

> ⚠️ **Cuidado**
> Partida no importa Flutter, ni GPS, ni cámara. Recibe la distancia YA calculada. Por eso las diez pruebas corren en milisegundos y sin ningún dispositivo.

> 🏆 **RETO PARA TI**
> Agrega una racha: capturar dos personajes seguidos sin fallar suma 25 puntos extra. Escribe primero la prueba. ¿Dónde guardarías el contador de racha?

🏅 **INSIGNIA DESBLOQUEADA: Diseñador de reglas**

Ronda 8 · Guardar el progreso (álbum)

Un contrato y dos implementaciones

El juego necesita recordar qué personajes ya atrapaste y tu mejor puntaje, aunque cierres la app. Definimos un contrato (AlmacenColeccion) con cuatro operaciones y dos implementaciones:

- AlmacenMemoria: guarda en variables. Sirve para las pruebas y para el modo demo (para no ensuciar tu progreso real).
- AlmacenPrefs: guarda en el teléfono con shared_preferences.

📄 **lib/dominio/almacen.dart**

```
/// Contrato para guardar el progreso. Solo guardamos ids y puntaje:
/// NUNCA coordenadas del jugador (privacidad).
abstract class AlmacenColeccion {
  Future<Set<String>> cargarCapturados();
  Future<void> guardarCapturados(Set<String> ids);
  Future<int> cargarMejorPuntaje();
  Future<void> guardarMejorPuntaje(int puntos);
}

/// En memoria: para pruebas y para el modo demo (no ensucia el progreso real).
class AlmacenMemoria implements AlmacenColeccion {
  Set<String> _ids = {};
  int _mejor = 0;

  @override
  Future<Set<String>> cargarCapturados() async => {..._ids};
  @override
  Future<void> guardarCapturados(Set<String> ids) async => _ids = {...ids};
  @override
  Future<int> cargarMejorPuntaje() async => _mejor;
  @override
  Future<void> guardarMejorPuntaje(int puntos) async => _mejor = puntos;
}
```

📄 **lib/datos/almacen_prefs.dart**

```
import 'package:shared_preferences/shared_preferences.dart';

import '../dominio/almacen.dart';

/// Guarda el progreso en el teléfono (sin internet, sin servidores).
class AlmacenPrefs implements AlmacenColeccion {
  static const _claveIds = 'capturados';
  static const _claveMejor = 'mejor_puntaje';

  @override
  Future<Set<String>> cargarCapturados() async {
    final p = await SharedPreferences.getInstance();
    return (p.getStringList(_claveIds) ?? const <String>[]).toSet();
  }

  @override
  Future<void> guardarCapturados(Set<String> ids) async {
    final p = await SharedPreferences.getInstance();
    await p.setStringList(_claveIds, ids.toList());
  }

  @override
```

```
  Future<int> cargarMejorPuntaje() async =>
      (await SharedPreferences.getInstance()).getInt(_claveMejor) ?? 0;

  @override
  Future<void> guardarMejorPuntaje(int puntos) async =>
      (await SharedPreferences.getInstance()).setInt(_claveMejor, puntos);
}
```

> 💡 **Dato curioso / ejemplo**
> Es el mismo patrón de la guía de la nevería (Repositorio): el resto del juego pide "guarda esto" sin saber dónde. Mañana podrías guardar en un servidor y no tocarías la Partida ni el ViewModel.

> ⚠️ **Cuidado**
> PRIVACIDAD: solo guardamos IDs de personajes y un puntaje. NUNCA guardamos coordenadas del jugador. Lo que no se guarda no se puede filtrar. Si algún día agregas un ranking en línea, avisa al usuario y pide consentimiento.

🏅 **INSIGNIA DESBLOQUEADA: Guardián del álbum**

Ronda 9 · Fuentes de posición y brújula (reales y simuladas)

Igual que en la guía Dolores AR, el ViewModel solo conoce dos contratos: FuentePosicion (entrega coordenadas) y FuenteBrujula (entrega rumbos). Hay tres implementaciones de cada una: real (teléfono), simulada (modo demo) y falsa (pruebas).

📄 **lib/dominio/fuentes.dart**

```
import 'geo.dart';

/// Contratos: el resto de la app NO sabe si la posición viene del GPS real
/// o de un simulador. Esa ignorancia es lo que permite probar y hacer demos.
abstract class FuentePosicion {
  Stream<Coordenada> get posiciones;
}

abstract class FuenteBrujula {
  /// Rumbo en grados (0 = norte).
  Stream<double> get rumbos;
}
```

📄 **lib/datos/fuentes_reales.dart**

```
import 'package:flutter_compass/flutter_compass.dart';
import 'package:geolocator/geolocator.dart';

import '../dominio/excepciones.dart';
import '../dominio/fuentes.dart';
import '../dominio/geo.dart';

class PosicionGps implements FuentePosicion {
  @override
  Stream<Coordenada> get posiciones async* {
    if (!await Geolocator.isLocationServiceEnabled()) {
      throw const UbicacionApagadaException();
    }
    var permiso = await Geolocator.checkPermission();
    if (permiso == LocationPermission.denied) {
      permiso = await Geolocator.requestPermission();
    }
    if (permiso == LocationPermission.denied ||
        permiso == LocationPermission.deniedForever) {
      throw const PermisoUbicacionException();
    }
    yield* Geolocator.getPositionStream(
      locationSettings: const LocationSettings(
        accuracy: LocationAccuracy.best,
        distanceFilter: 2, // solo avisa si te moviste 2 m o más
      ),
    ).map((p) => Coordenada(p.latitude, p.longitude));
  }
}

class BrujulaReal implements FuenteBrujula {
  @override
  Stream<double> get rumbos {
    final eventos = FlutterCompass.events;
    if (eventos == null) return Stream.error(const SinBrujulaException());
    return eventos.where((e) => e.heading != null).map((e) => normalizar360(e.heading!));
  }
}
```

La novedad está en el modo demo: PosicionSimulada permite CAMINAR. Usa la función destino que acabamos de probar:

📄 **lib/datos/fuentes_simuladas.dart**

```
import 'dart:async';

import '../dominio/fuentes.dart';
import '../dominio/geo.dart';

/// Modo demo: el jugador "camina" con botones dentro del salón de clases.
class PosicionSimulada implements FuentePosicion {
  Coordenada _actual;
  final _controlador = StreamController<Coordenada>.broadcast();
  PosicionSimulada(this._actual);

  @override
  Stream<Coordenada> get posiciones async* {
    yield _actual;
    yield* _controlador.stream;
  }

  /// Camina [metros] con el [rumbo] dado (negativo = retrocede).
  void avanzar(double rumbo, double metros) {
    _actual = metros >= 0
        ? destino(_actual, rumbo, metros)
        : destino(_actual, rumbo + 180, -metros);
    _controlador.add(_actual);
  }

  void cerrar() => _controlador.close();
}

class BrujulaSimulada implements FuenteBrujula {
  final _controlador = StreamController<double>.broadcast();
  double _ultimo = 0;

  @override
  Stream<double> get rumbos async* {
    yield _ultimo;
    yield* _controlador.stream;
  }

  void girar(double grados) {
    _ultimo = grados;
    _controlador.add(grados);
  }

  void cerrar() => _controlador.close();
}
```

> 💡 **Dato curioso / ejemplo**
> Retroceder es avanzar en dirección contraria: rumbo + 180°. Dos líneas, sin fórmulas nuevas, gracias a que destino ya está probada. Reutilizar piezas probadas es la gran ventaja de construir con TDD.

> ⚠️ **Cuidado**
> El GPS es de los sensores que más batería consumen. distanceFilter: 2 hace que solo avise cuando te moviste 2 m. Y recuerda cancelar las suscripciones en dispose().

🏅 **INSIGNIA DESBLOQUEADA: Simulador de caminatas**

Ronda 10 · El ViewModel del juego

El ViewModel junta todo: escucha GPS y brújula, recalcula lo que se ve, calcula la temperatura y el radar, y ejecuta las capturas guardando el progreso. Lo probamos con fuentes falsas controladas por StreamController y una semilla fija: el escenario es siempre el mismo.

Ciclo 10.1 · ROJO

📄 **test/rally_viewmodel_test.dart**

```
import 'dart:async';
import 'dart:math';

import 'package:dolores_rally/dominio/almacen.dart';
import 'package:dolores_rally/dominio/catalogo.dart';
import 'package:dolores_rally/dominio/excepciones.dart';
import 'package:dolores_rally/dominio/fuentes.dart';
import 'package:dolores_rally/dominio/geo.dart';
import 'package:dolores_rally/dominio/partida.dart';
import 'package:dolores_rally/dominio/radar.dart';
import 'package:dolores_rally/presentacion/rally_viewmodel.dart';
import 'package:flutter_test/flutter_test.dart';

import 'ayudas.dart';

class PosicionFalsa implements FuentePosicion {
  final c = StreamController<Coordenada>.broadcast();
  @override
  Stream<Coordenada> get posiciones => c.stream;
}

class BrujulaFalsa implements FuenteBrujula {
  final c = StreamController<double>.broadcast();
  @override
  Stream<double> get rumbos => c.stream;
}

void main() {
  late PosicionFalsa gps;
  late BrujulaFalsa brujula;
  late AlmacenMemoria almacen;

  Future<RallyViewModel> crear() async {
    gps = PosicionFalsa();
    brujula = BrujulaFalsa();
    almacen = AlmacenMemoria();
    final vm = RallyViewModel(
      posicion: gps,
      brujula: brujula,
      almacen: almacen,
      personajes: personajes,
      centro: origen,
      random: Random(42), // semilla fija: el escenario es siempre el mismo
    );
    await vm.iniciar();
    return vm;
  }

  test('mientras falte GPS o brújula, está esperando', () async {
    final vm = await crear();
    expect(vm.esperando, true);
```

```
  gps.c.add(origen);
  await pumpEventQueue();
  expect(vm.esperando, true);
  vm.dispose();
});

test('con GPS y brújula calcula visibles, radar y temperatura', () async {
  final vm = await crear();
  gps.c.add(origen);
  brujula.c.add(0);
  await pumpEventQueue();
  expect(vm.esperando, false);
  expect(vm.temperatura, isNotNull);
  expect(vm.puntosRadar, isNotEmpty);
  vm.dispose();
});

test('el más cercano es el que marca la temperatura', () async {
  final vm = await crear();
  final objetivo = vm.partida.todas.first;
  gps.c.add(destino(objetivo.coordenada, 90, 8)); // a 8 m de ese personaje
  brujula.c.add(0);
  await pumpEventQueue();
  expect(vm.masCercano!.personaje.id, objetivo.personaje.id);
  expect(vm.temperatura, Temperatura.ardiente);
  vm.dispose();
});

test('capturar con respuesta correcta suma puntos y guarda el progreso', () async {
  final vm = await crear();
  final objetivo = vm.partida.todas.first;
  gps.c.add(destino(objetivo.coordenada, 0, 5));
  brujula.c.add(0);
  await pumpEventQueue();

  final correcta = objetivo.personaje.pregunta.correcta;
  final r = await vm.capturar(objetivo.personaje.id, correcta);
  expect(r, isA<Capturado>());
  expect(vm.partida.puntos, 100);
  expect(await almacen.cargarCapturados(), {objetivo.personaje.id});
  expect(await almacen.cargarMejorPuntaje(), 100);
  vm.dispose();
});

test('capturar estando lejos lanza LejosException y no guarda nada', () async {
  final vm = await crear();
  gps.c.add(origen);
  brujula.c.add(0);
  await pumpEventQueue();
  final objetivo = vm.partida.todas.first;
  expect(() => vm.capturar(objetivo.personaje.id, 0), throwsA(isA<LejosException>()));
  expect(await almacen.cargarCapturados(), isEmpty);
  vm.dispose();
});

test('al iniciar restaura el progreso guardado', () async {
  gps = PosicionFalsa();
  brujula = BrujulaFalsa();
  almacen = AlmacenMemoria();
  await almacen.guardarCapturados({'hidalgo'});
  final vm = RallyViewModel(
    posicion: gps, brujula: brujula, almacen: almacen,
    personajes: personajes, centro: origen, random: Random(1),
```

```
    );
    await vm.iniciar();
    expect(vm.partida.estaCapturado('hidalgo'), true);
    expect(vm.partida.pendientes.length, personajes.length - 1);
    vm.dispose();
  });

  test('lejosDeDolores avisa cuando estás a más de 3 km del centro', () async {
    final vm = await crear();
    gps.c.add(destino(origen, 0, 5000));
    brujula.c.add(0);
    await pumpEventQueue();
    expect(vm.lejosDeDolores, true);
    vm.dispose();
  });

  test('un error de permiso se muestra con mensaje amable', () async {
    final vm = await crear();
    gps.c.addError(const PermisoUbicacionException());
    await pumpEventQueue();
    expect(vm.error, contains('permiso'));
    vm.dispose();
  });

  test('un error desconocido NO filtra detalles internos', () async {
    final vm = await crear();
    brujula.c.addError(StateError('fallo en /sensores/mag.dart'));
    await pumpEventQueue();
    expect(vm.error, isNot(contains('/sensores')));
    expect(vm.error, contains('inesperado'));
    vm.dispose();
  });
}
```

Observa el truco de la prueba "el más cercano marca la temperatura": en lugar de adivinar dónde quedó el personaje, la prueba LEE su posición (vm.partida.todas.first) y se coloca a 8 metros con destino(). Escribimos pruebas que se adaptan a los datos generados.

Ciclo 10.2 · VERDE

📄 **lib/presentacion/rally_viewmodel.dart**

```
import 'dart:async';
import 'dart:math' as math;

import 'package:flutter/foundation.dart';

import '../dominio/almacen.dart';
import '../dominio/aparicion.dart';
import '../dominio/excepciones.dart';
import '../dominio/fuentes.dart';
import '../dominio/geo.dart';
import '../dominio/motor_ar.dart';
import '../dominio/partida.dart';
import '../dominio/personaje.dart';
import '../dominio/radar.dart';

class RallyViewModel extends ChangeNotifier {
  static const double limiteLejos = 3000; // más lejos que esto: no estás en Dolores

  final FuentePosicion _posicion;
```

```
  final FuenteBrujula _brujula;
  final AlmacenColeccion _almacen;
  final Coordenada centro;
  final double campoVision;
  final double radioMax;
  late final Partida partida;

  RallyViewModel({
    required FuentePosicion posicion,
    required FuenteBrujula brujula,
    required AlmacenColeccion almacen,
    required List<Personaje> personajes,
    required this.centro,
    required math.Random random,
    this.campoVision = 60,
    this.radioMax = 600,
  })  : _posicion = posicion,
        _brujula = brujula,
        _almacen = almacen {
    partida = Partida(
      generarApariciones(centro: centro, personajes: personajes, random: random, radioMax:
500),
      reubicar: () => puntoAleatorio(centro, 60, 500, random),
    );
  }

  Coordenada? usuario;
  double? rumbo;
  List<Visible> visibles = [];
  String? error;

  StreamSubscription<Coordenada>? _subPos;
  StreamSubscription<double>? _subBru;

  bool get esperando => usuario == null || rumbo == null;

  bool get lejosDeDolores =>
      usuario != null && distanciaMetros(usuario!, centro) > limiteLejos;

  /// Personaje pendiente más cercano (para el "frío o caliente").
  Aparicion? get masCercano {
    final u = usuario;
    if (u == null || partida.pendientes.isEmpty) return null;
    final lista = [...partida.pendientes];
    lista.sort((a, b) =>
        distanciaMetros(u, a.coordenada).compareTo(distanciaMetros(u, b.coordenada)));
    return lista.first;
  }

  double? get distanciaAlMasCercano {
    final c = masCercano;
    return (c == null || usuario == null) ? null : distanciaMetros(usuario!, c.coordenada);
  }

  Temperatura? get temperatura =>
      distanciaAlMasCercano == null ? null : temperaturaPorDistancia(distanciaAlMasCercano!);

  /// Posiciones (x, y) de los pendientes dentro del radar.
  List<(double, double)> get puntosRadar {
    final u = usuario, r = rumbo;
    if (u == null || r == null) return const [];
    return [
      for (final a in partida.pendientes)
```

```
        if (puntoEnRadar(usuario: u, rumbo: r, objetivo: a.coordenada, alcance: radioMax)
            case final p?)
          p,
    ];
  }

  Future<void> iniciar() async {
    partida.restaurar(await _almacen.cargarCapturados());
    _subPos = _posicion.posiciones.listen((p) {
      usuario = p;
      _recalcular();
    }, onError: _alFallar);
    _subBru = _brujula.rumbos.listen((g) {
      rumbo = rumbo == null ? g : suavizarAngulo(rumbo!, g);
      _recalcular();
    }, onError: _alFallar);
    notifyListeners();
  }

  void _alFallar(Object e) {
    error = mensajeSeguro(e);
    notifyListeners();
  }

  void _recalcular() {
    final u = usuario, r = rumbo;
    if (u != null && r != null) {
      visibles = calcularVisibles(
        usuario: u,
        rumbo: r,
        apariciones: partida.pendientes,
        campoVision: campoVision,
        radioMax: radioMax,
      );
    }
    notifyListeners();
  }

  /// Intenta atrapar. Lanza LejosException si no estás a menos de 15 m.
  Future<ResultadoCaptura> capturar(String id, int opcion) async {
    final u = usuario;
    final aparicion = partida.todas.where((a) => a.personaje.id == id).firstOrNull;
    final d = (u == null || aparicion == null)
        ? double.infinity
        : distanciaMetros(u, aparicion.coordenada);
    final resultado = partida.capturar(id, opcion, d);
    if (resultado is Capturado) {
      await _almacen.guardarCapturados(partida.capturados);
      if (partida.puntos > await _almacen.cargarMejorPuntaje()) {
        await _almacen.guardarMejorPuntaje(partida.puntos);
      }
    }
    _recalcular();
    return resultado;
  }

  @override
  void dispose() {
    _subPos?.cancel();
    _subBru?.cancel();
    super.dispose();
  }
}
```

```
$ flutter test
```

Detalles clave

- iniciar() es async: primero restaura el progreso guardado, luego escucha los sensores.
- capturar() calcula la distancia real al personaje y delega las reglas a Partida; si capturó, guarda el progreso y el mejor puntaje.
- lejosDeDolores avisa cuando estás a más de 3 km del centro (por ejemplo, jugando desde tu casa en otra ciudad).
- puntosRadar usa un pattern match con "case final p?" para ignorar a los que están fuera de alcance.

> 🔵 **REFACTOR: ordena sin romper nada**
> Todas las pruebas (geo, azar, radar, motor, partida, ViewModel) corren sin teléfono, sin cámara y sin GPS.
> Lo que aún no hemos probado automáticamente es lo visual: de eso se encarga la prueba de campo.

🏅 **INSIGNIA DESBLOQUEADA: Director del juego**

Ronda 11 · Las pantallas

Paso 11.1 · El fondo de cámara

Muestra la cámara trasera. Si no hay cámara o el permiso se niega, usa un degradado: el juego sigue funcionando (ideal para emuladores y para el salón de clases).

📄 **lib/presentacion/fondo_camara.dart**

```
import 'package:camera/camera.dart';
import 'package:flutter/material.dart';

/// Muestra la cámara trasera. Si no hay cámara o el permiso se niega
/// (emuladores, computadoras), usa un degradado: la app nunca se rompe.
class FondoCamara extends StatefulWidget {
  const FondoCamara({super.key});
  @override
  State<FondoCamara> createState() => _FondoCamaraState();
}

class _FondoCamaraState extends State<FondoCamara> {
  CameraController? _controlador;
  bool _fallo = false;

  @override
  void initState() {
    super.initState();
    _iniciar();
  }

  Future<void> _iniciar() async {
    try {
      final camaras = await availableCameras();
      final trasera = camaras.firstWhere(
        (c) => c.lensDirection == CameraLensDirection.back,
        orElse: () => camaras.first,
      );
      final c = CameraController(trasera, ResolutionPreset.medium, enableAudio: false);
      await c.initialize();
      if (!mounted) {
        await c.dispose();
        return;
      }
      setState(() => _controlador = c);
    } catch (_) {
      if (mounted) setState(() => _fallo = true);
    }
  }

  @override
  void dispose() {
    _controlador?.dispose(); // liberar la cámara es obligatorio
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final c = _controlador;
    if (c != null && c.value.isInitialized) {
      final t = c.value.previewSize!; // viene en horizontal: se invierte en vertical
      return SizedBox.expand(
        child: FittedBox(
```

```
          fit: BoxFit.cover,
          child: SizedBox(width: t.height, height: t.width, child: CameraPreview(c)),
        ),
      );
    }
    return Container(
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
          colors: [Color(0xFF1B2A49), Color(0xFF4A6FA5), Color(0xFFD9A066)],
        ),
      ),
      child: _fallo
          ? const Align(
              alignment: Alignment.bottomCenter,
              child: Padding(
                padding: EdgeInsets.all(8),
                child: Text('Sin cámara: usando fondo simulado',
                    style: TextStyle(color: Colors.white70, fontSize: 12)),
              ),
            )
          : null,
    );
  }
}
```

Paso 11.2 · El radar con CustomPainter

CustomPainter te da un lienzo (canvas) para dibujar con código: círculos, líneas, rutas. Dibujamos el fondo oscuro, dos círculos concéntricos, la cruz, el triángulo del jugador y un punto naranja por cada personaje pendiente. shouldRepaint le dice a Flutter cuándo vale la pena volver a pintar.

📄 **lib/presentacion/radar_widget.dart**

```
import 'package:flutter/material.dart';

/// Radar circular: tú estás en el centro y arriba es hacia donde miras.
/// Cada punto es un personaje pendiente (x, y entre -1 y 1).
class RadarWidget extends StatelessWidget {
  final List<(double, double)> puntos;
  final double tamano;
  const RadarWidget({super.key, required this.puntos, this.tamano = 110});

  @override
  Widget build(BuildContext context) => SizedBox(
        width: tamano,
        height: tamano,
        child: CustomPaint(painter: _RadarPainter(puntos)),
      );
}

class _RadarPainter extends CustomPainter {
  final List<(double, double)> puntos;
  _RadarPainter(this.puntos);

  @override
  void paint(Canvas canvas, Size size) {
    final centro = size.center(Offset.zero);
    final radio = size.width / 2;
```

```
    canvas.drawCircle(centro, radio, Paint()..color = Colors.black54);
    final linea = Paint()
      ..color = Colors.greenAccent.withOpacity(0.6)
      ..style = PaintingStyle.stroke
      ..strokeWidth = 1;
    canvas.drawCircle(centro, radio, linea);
    canvas.drawCircle(centro, radio / 2, linea);
    canvas.drawLine(Offset(centro.dx, 0), Offset(centro.dx, size.height), linea);
    canvas.drawLine(Offset(0, centro.dy), Offset(size.width, centro.dy), linea);

    // Triángulo del jugador (siempre apunta arriba).
    final yo = Path()
      ..moveTo(centro.dx, centro.dy - 7)
      ..lineTo(centro.dx - 5, centro.dy + 5)
      ..lineTo(centro.dx + 5, centro.dy + 5)
      ..close();
    canvas.drawPath(yo, Paint()..color = Colors.white);

    for (final (x, y) in puntos) {
      canvas.drawCircle(
        Offset(centro.dx + x * (radio - 6), centro.dy + y * (radio - 6)),
        5,
        Paint()..color = Colors.orangeAccent,
      );
    }
  }

  @override
  bool shouldRepaint(_RadarPainter viejo) => viejo.puntos != puntos;
}
```

> 💡 **Dato curioso / ejemplo**
> La posición final de cada punto es: centro + (x, y) × (radio − 6). El "−6" evita que el punto se corte en el borde. Es la conversión del mundo matemático (de −1 a 1) al mundo de píxeles.

Paso 11.3 · La pantalla del juego

📄 **lib/presentacion/pantalla_rally.dart**

```
import 'package:flutter/material.dart';

import '../dominio/excepciones.dart';
import '../dominio/geo.dart';
import '../dominio/motor_ar.dart';
import '../dominio/partida.dart';
import '../dominio/personaje.dart';
import '../dominio/radar.dart';
import 'fondo_camara.dart';
import 'radar_widget.dart';
import 'rally_viewmodel.dart';

class PantallaRally extends StatefulWidget {
  final RallyViewModel Function() crearVm;
  final Widget? controlesDemo;
  const PantallaRally({super.key, required this.crearVm, this.controlesDemo});

  @override
  State<PantallaRally> createState() => _PantallaRallyState();
}

class _PantallaRallyState extends State<PantallaRally> {
```

```
late final RallyViewModel vm = widget.crearVm();

@override
void initState() {
  super.initState();
  vm.iniciar();
}

@override
void dispose() {
  vm.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: AnimatedBuilder(
      animation: vm,
      builder: (_, __) => Stack(fit: StackFit.expand, children: [
        const FondoCamara(),
        LayoutBuilder(
          builder: (_, caja) => Stack(children: [
            for (final v in vm.visibles) _personaje(v, caja),
          ]),
        ),
        SafeArea(
          child: Align(
            alignment: Alignment.topLeft,
            child: Padding(padding: const EdgeInsets.all(8), child: _marcador()),
          ),
        ),
        SafeArea(
          child: Align(
            alignment: Alignment.topRight,
            child: Padding(
              padding: const EdgeInsets.all(8),
              child: RadarWidget(puntos: vm.puntosRadar),
            ),
          ),
        ),
        SafeArea(child: Align(alignment: Alignment.bottomCenter, child: _panelInferior())),
      ]),
    ),
  );
}

Widget _marcador() {
  final texto = vm.error ??
      (vm.esperando
          ? 'Buscando GPS y brújula…'
          : vm.lejosDeDolores
              ? '📍 Estás lejos de Dolores Hidalgo: usa el modo demo'
              : '${nombreCardinal(vm.rumbo!)} ${vm.rumbo!.round()}°\n'
                  '🏅 ${vm.partida.capturados.length}/${vm.partida.todas.length} · '
                  '⭐ ${vm.partida.puntos}\n'
                  '${vm.temperatura?.texto ?? '🎉 ¡Los atrapaste a todos!'}');
  return Container(
    constraints: const BoxConstraints(maxWidth: 220),
    padding: const EdgeInsets.all(10),
    decoration: BoxDecoration(
      color: vm.error != null ? Colors.red.shade800 : Colors.black54,
      borderRadius: BorderRadius.circular(14),
```

```
      ),
      child: Text(texto, style: const TextStyle(color: Colors.white)),
    );
  }

  Widget _personaje(Visible v, BoxConstraints caja) {
    const ancho = 130.0;
    final alAlcance = v.distancia <= Partida.radioCaptura;
    return Positioned(
      left: v.x * caja.maxWidth - ancho / 2,
      top: caja.maxHeight * (0.50 - 0.28 * v.profundidad),
      width: ancho,
      child: Transform.scale(
        scale: v.escala,
        child: GestureDetector(
          onTap: () => _intentar(v),
          child: Column(mainAxisSize: MainAxisSize.min, children: [
            Container(
              padding: const EdgeInsets.all(10),
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                color: Colors.white.withOpacity(0.85),
                border: Border.all(color: alAlcance ? Colors.greenAccent : Colors.white,
width: 4),
              ),
              child: Text(v.aparicion.personaje.emoji, style: const TextStyle(fontSize: 44)),
            ),
            Container(
              margin: const EdgeInsets.only(top: 4),
              padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 2),
              decoration: BoxDecoration(color: Colors.black54, borderRadius:
BorderRadius.circular(10)),
              child: Text(
                alAlcance ? '¡Tócame!' : formatearDistancia(v.distancia),
                style: const TextStyle(color: Colors.white, fontSize: 12),
              ),
            ),
          ]),
        ),
      ),
    );
  }

  Widget _panelInferior() => Column(mainAxisSize: MainAxisSize.min, children: [
        if (widget.controlesDemo != null) widget.controlesDemo!,
      ]);

  Future<void> _intentar(Visible v) async {
    final p = v.aparicion.personaje;
    if (v.distancia > Partida.radioCaptura) {
      _aviso(LejosException(v.distancia).toString());
      return;
    }
    final opcion = await showDialog<int>(
      context: context,
      barrierDismissible: false,
      builder: (_) => _DialogoPregunta(personaje: p),
    );
    if (opcion == null) return;
    try {
      final r = await vm.capturar(p.id, opcion);
      switch (r) {
        case Capturado(:final puntos):
```

```
          _aviso('🎉 ¡Atrapaste a ${p.nombre}! +$puntos puntos\n${p.bio}');
        case Fallo(:final intentosRestantes):
          _aviso('❌ Respuesta incorrecta. Te queda $intentosRestantes intento.');
        case SeEscapo():
          _aviso('💨 ¡${p.nombre} se escapó! Búscalo en otro lugar.');
      }
    } catch (e) {
      _aviso(mensajeSeguro(e));
    }
  }

  void _aviso(String texto) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(texto), duration: const Duration(seconds: 4)),
    );
  }
}

class _DialogoPregunta extends StatelessWidget {
  final Personaje personaje;
  const _DialogoPregunta({required this.personaje});

  @override
  Widget build(BuildContext context) {
    final q = personaje.pregunta;
    return AlertDialog(
      title: Text('${personaje.emoji} ${personaje.nombre}'),
      content: Column(mainAxisSize: MainAxisSize.min, crossAxisAlignment:
CrossAxisAlignment.start, children: [
        Text(q.texto, style: const TextStyle(fontWeight: FontWeight.bold)),
        const SizedBox(height: 12),
        for (var i = 0; i < q.opciones.length; i++)
          Padding(
            padding: const EdgeInsets.only(bottom: 6),
            child: OutlinedButton(
              onPressed: () => Navigator.pop(context, i),
              child: Text(q.opciones[i], textAlign: TextAlign.center),
            ),
          ),
      ]),
    );
  }
}
```

Puntos de diseño

- Capas con Stack: cámara → personajes → marcador → radar → controles.
- Cuando un personaje está a 15 m o menos, su círculo se vuelve verde y dice "¡Tócame!": retroalimentación inmediata.
- El switch sobre ResultadoCaptura es exhaustivo: si agregas un resultado nuevo, el compilador te avisa.
- Todos los errores pasan por mensajeSeguro: el jugador ve mensajes amables, nunca detalles técnicos.

Paso 11.4 · Álbum, inicio y main

📄 **lib/presentacion/pantalla_coleccion.dart**

```
import 'package:flutter/material.dart';

import '../dominio/almacen.dart';
```

```dart
import '../dominio/personaje.dart';

/// Álbum: muestra los personajes atrapados y deja en "?" los pendientes.
class PantallaColeccion extends StatelessWidget {
  final AlmacenColeccion almacen;
  final List<Personaje> personajes;
  const PantallaColeccion({super.key, required this.almacen, required this.personajes});

  Future<(Set<String>, int)> _cargar() async =>
      (await almacen.cargarCapturados(), await almacen.cargarMejorPuntaje());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Mi álbum')),
      body: FutureBuilder<(Set<String>, int)>(
        future: _cargar(),
        builder: (context, snap) {
          if (!snap.hasData) return const Center(child: CircularProgressIndicator());
          final (ids, mejor) = snap.data!;
          return Column(children: [
            Padding(
              padding: const EdgeInsets.all(12),
              child: Text('🏅 ${ids.length}/${personajes.length}  ·  ⭐ Mejor puntaje:
$mejor'),
            ),
            Expanded(
              child: GridView.count(
                crossAxisCount: 2,
                padding: const EdgeInsets.all(12),
                mainAxisSpacing: 12,
                crossAxisSpacing: 12,
                children: [
                  for (final p in personajes)
                    _Carta(personaje: p, desbloqueado: ids.contains(p.id)),
                ],
              ),
            ),
          ]);
        },
      ),
    );
  }
}

class _Carta extends StatelessWidget {
  final Personaje personaje;
  final bool desbloqueado;
  const _Carta({required this.personaje, required this.desbloqueado});

  @override
  Widget build(BuildContext context) {
    return Card(
      color: desbloqueado ? null : Colors.grey.shade300,
      child: InkWell(
        onTap: desbloqueado
            ? () => showDialog<void>(
                  context: context,
                  builder: (_) => AlertDialog(
                    title: Text('${personaje.emoji} ${personaje.nombre}'),
                    content: Text(personaje.bio),
                  ),
                )
```

```
            : null,
        child: Padding(
          padding: const EdgeInsets.all(8),
          child: Column(mainAxisAlignment: MainAxisAlignment.center, children: [
            Text(desbloqueado ? personaje.emoji : '❓', style: const TextStyle(fontSize: 48)),
            const SizedBox(height: 6),
            Text(desbloqueado ? personaje.nombre : 'Por descubrir',
                textAlign: TextAlign.center, style: const TextStyle(fontWeight:
FontWeight.bold)),
          ]),
        ),
      ),
    );
  }
}
```

📄 lib/presentacion/pantalla_inicio.dart

```
import 'dart:math' as math;

import 'package:flutter/material.dart';

import '../datos/almacen_prefs.dart';
import '../datos/fuentes_reales.dart';
import '../datos/fuentes_simuladas.dart';
import '../dominio/almacen.dart';
import '../dominio/catalogo.dart';
import 'pantalla_coleccion.dart';
import 'pantalla_rally.dart';
import 'rally_viewmodel.dart';

class PantallaInicio extends StatelessWidget {
  const PantallaInicio({super.key});

  void _jugarReal(BuildContext context) {
    Navigator.push(context, MaterialPageRoute<void>(
      builder: (_) => PantallaRally(
        crearVm: () => RallyViewModel(
          posicion: PosicionGps(),
          brujula: BrujulaReal(),
          almacen: AlmacenPrefs(),
          personajes: personajes,
          centro: plazaPrincipal,
          random: math.Random(), // sin semilla: cada partida es distinta
        ),
      ),
    ));
  }

  void _jugarDemo(BuildContext context) {
    final posicion = PosicionSimulada(plazaPrincipal);
    final brujula = BrujulaSimulada();
    Navigator.push(context, MaterialPageRoute<void>(
      builder: (_) => PantallaRally(
        crearVm: () => RallyViewModel(
          posicion: posicion,
          brujula: brujula,
          almacen: AlmacenMemoria(), // el demo no toca tu progreso real
          personajes: personajes,
          centro: plazaPrincipal,
          random: math.Random(),
        ),
        controlesDemo: _ControlesDemo(posicion: posicion, brujula: brujula),
```

```
      ),
    ));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Rally de la Independencia AR')),
      body: ListView(padding: const EdgeInsets.all(16), children: [
        Text('Atrapa a los personajes escondidos en Dolores Hidalgo',
            style: Theme.of(context).textTheme.headlineSmall),
        const SizedBox(height: 8),
        const Text('Usa el radar y el "frío o caliente" para encontrarlos. '
            'Acércate a menos de 15 m y responde su pregunta para atraparlos.'),
        const SizedBox(height: 12),
        Card(
          color: Colors.amber.shade100,
          child: const Padding(
            padding: EdgeInsets.all(12),
            child: Text('⚠️ Mira a tu alrededor, cuida tu paso y no cruces la calle viendo el
teléfono.'),
          ),
        ),
        const SizedBox(height: 16),
        FilledButton.icon(
          onPressed: () => _jugarReal(context),
          icon: const Icon(Icons.explore),
          label: const Text('Jugar en Dolores Hidalgo (GPS real)'),
        ),
        const SizedBox(height: 10),
        OutlinedButton.icon(
          onPressed: () => _jugarDemo(context),
          icon: const Icon(Icons.school),
          label: const Text('Modo demo (salón de clases)'),
        ),
        const SizedBox(height: 10),
        OutlinedButton.icon(
          onPressed: () => Navigator.push(context, MaterialPageRoute<void>(
            builder: (_) => PantallaColeccion(almacen: AlmacenPrefs(), personajes:
personajes),
          )),
          icon: const Icon(Icons.collections_bookmark),
          label: const Text('Mi álbum'),
        ),
      ]),
    );
  }
}

/// Deslizador para girar y botones para "caminar" en el salón.
class _ControlesDemo extends StatefulWidget {
  final PosicionSimulada posicion;
  final BrujulaSimulada brujula;
  const _ControlesDemo({required this.posicion, required this.brujula});
  @override
  State<_ControlesDemo> createState() => _ControlesDemoState();
}

class _ControlesDemoState extends State<_ControlesDemo> {
  double _grados = 0;

  @override
  Widget build(BuildContext context) {
```

```
    return Container(
      color: Colors.black54,
      padding: const EdgeInsets.symmetric(horizontal: 8),
      child: Column(mainAxisSize: MainAxisSize.min, children: [
        Row(children: [
          const Text('Gira', style: TextStyle(color: Colors.white)),
          Expanded(
            child: Slider(
              min: 0,
              max: 359,
              value: _grados,
              onChanged: (g) {
                setState(() => _grados = g);
                widget.brujula.girar(g);
              },
            ),
          ),
          Text('${_grados.round()}°', style: const TextStyle(color: Colors.white)),
        ]),
        Row(mainAxisAlignment: MainAxisAlignment.spaceEvenly, children: [
          OutlinedButton(
            onPressed: () => widget.posicion.avanzar(_grados, -10),
            child: const Text('⬇ 10 m'),
          ),
          FilledButton(
            onPressed: () => widget.posicion.avanzar(_grados, 10),
            child: const Text('⬆ Avanzar 10 m'),
          ),
          OutlinedButton(
            onPressed: () => widget.posicion.avanzar(_grados, 2),
            child: const Text('+2 m'),
          ),
        ]),
      ]),
    );
  }
}
```

📄 **lib/main.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import 'presentacion/pantalla_inicio.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  SystemChrome.setPreferredOrientations([DeviceOrientation.portraitUp]);
  runApp(MaterialApp(
    title: 'Rally de la Independencia AR',
    theme: ThemeData(colorSchemeSeed: Colors.green, useMaterial3: true),
    home: const PantallaInicio(),
  ));
}
```

Paso 11.5 · Primera partida en modo demo

```
$ flutter run
```

19. Toca "Modo demo (salón de clases)": estás en el Jardín Principal.

20. Gira con el deslizador y observa el radar: los puntos naranjas rotan alrededor de ti.
21. Fíjate en el letrero de temperatura (🥶 Helado ... 🌋 Ardiendo).
22. Gira hasta ver un personaje flotando; apúntalo y usa "Avanzar 10 m" varias veces. El letrero se calienta.
23. Cuando el círculo se ponga verde (≤ 15 m), tócalo y responde la pregunta.
24. Falla a propósito dos veces: observa cómo el personaje huye y reaparece lejos.
25. Atrapa a los 6. Regresa al inicio y revisa "Mi álbum" (en demo el progreso no se guarda; en modo real sí).

> 💡 **Dato curioso / ejemplo**
> Todo el salón puede jugar a la vez: cada equipo con su teléfono en modo demo, compitiendo por quién atrapa a los 6 personajes primero. Como el azar usa Random() sin semilla, cada equipo tiene un mapa distinto.

🏅 **INSIGNIA DESBLOQUEADA: Mago del juego**

Ronda 12 · Prueba de campo, balanceo y seguridad

12.1 · Playtesting: jugar con jugadores reales

Un juego no está terminado cuando "funciona", sino cuando es divertido. Organiza una sesión de prueba con 4 o 5 compañeros en el Jardín Principal y observa SIN ayudar. Anota:

- ¿Cuánto tardó cada persona en atrapar al primer personaje? (Si es más de 10 minutos, está muy difícil.)
- ¿Hubo momentos de frustración? ¿Dónde? (Por ejemplo: estar a 10 m y que no se active.)
- ¿Entendieron el radar sin explicación?
- ¿Aprendieron algo de historia? Pregúntales después qué recuerdan.

12.2 · Qué números ajustar

- Radio de captura (Partida.radioCaptura): si el GPS es impreciso, sube de 15 a 20 o 25 m.
- Radio de aparición (60 a 500 m): si el recorrido es demasiado largo, baja el máximo a 300 m.
- Umbrales de temperatura: ajusta para que "caliente" aparezca con tiempo para reaccionar.
- Campo de visión (60°): si las etiquetas no coinciden con la dirección real, prueba 55° o 70°.
- Alfa del filtro de brújula (0.2): más bajo = más suave pero con retraso.

> ⚠️ **Cuidado**
> Cada vez que cambies una constante, vuelve a correr flutter test. Las pruebas que usan el valor fijo "15" fallarán y te recordarán actualizar el diseño en TODAS partes (por eso conviene usar la constante Partida.radioCaptura en lugar de escribir 15).

12.3 · Seguridad física

- Un juego de RA invita a caminar mirando la pantalla. La pantalla de inicio ya incluye un aviso; manténlo siempre visible.
- Jueguen en grupos y establezcan "zonas seguras": evitar cruzar calles, rieles o zonas de obra.
- Diseña la dinámica para que se vea el entorno: el radar permite buscar sin mirar la cámara todo el tiempo.
- Si los participantes son menores de edad, requiere la supervisión de un adulto y evita horarios de poca luz.

12.4 · Privacidad y trampas

- La ubicación nunca sale del teléfono: no hay servidor. El progreso guardado contiene solo ids y puntaje.
- Pide el permiso mínimo y explica para qué sirve con textos claros.
- Si algún día agregas un ranking en línea, enfrenta el problema de las "trampas": existen apps que falsifican el GPS. Un servidor honesto no debe confiar ciegamente en las coordenadas que manda el teléfono; se validan velocidad y saltos imposibles. Es un gran tema de investigación para el equipo.

> 💡 **Dato curioso / ejemplo**

> Imagina un jugador que "se teletransporta" 5 km en un segundo. La regla anti-trampa más sencilla es calcular la velocidad entre dos lecturas (distancia / tiempo) y rechazar las imposibles. ¿Con qué función de geo.dart la calcularías? ¡Con distanciaMetros!

🏅 **INSIGNIA DESBLOQUEADA: Explorador responsable**

Cierre · Lo que lograste

Checklist de calidad

- Las reglas del juego viven en dominio/ y no importan Flutter ni plugins.
- El azar está inyectado: las pruebas usan semilla fija y verifican propiedades.
- La función destino se probó en 9 direcciones (ida y vuelta).
- Los resultados de captura son una sealed class y el switch es exhaustivo.
- El radar y la temperatura tienen pruebas de orientación y de umbrales.
- El progreso guardado contiene solo ids y puntaje (nada de coordenadas).
- El modo demo permite jugar sin GPS ni brújula.
- La app no se rompe sin cámara, sin permiso o sin brújula.
- Se cancelan suscripciones y se libera la cámara en dispose().
- Hay aviso de seguridad física y se jugó con un grupo de prueba.

Preguntas para discutir en clase

26. ¿Por qué inyectar Random en lugar de crearlo dentro de generarApariciones?
27. ¿Qué pasaría con la distribución de los personajes si NO usáramos la raíz cuadrada?
28. ¿Por qué una prueba de "ida y vuelta" es mejor que probar valores calculados a mano?
29. ¿Qué ventaja da una sealed class sobre devolver un número o un String como resultado?
30. El GPS dice que estás a 18 m de un personaje pero lo ves delante de ti. ¿Qué dos números del juego ajustarías y por qué?
31. ¿Qué riesgos tiene un ranking en línea que recibe coordenadas del teléfono?

Retos finales por equipos

- Contrarreloj: modo de juego con 10 minutos para atrapar el mayor número de personajes (inyecta un reloj para probarlo).
- Pistas: gasta 20 puntos para que el juego diga el rumbo cardinal del personaje más cercano.
- Sonidos: un "pip" más rápido mientras te acercas (paquete audioplayers).
- Personajes en 3D: reemplaza el emoji por un modelo (investiga model_viewer_plus o flutter_unity_widget).
- Modo equipos: dos teléfonos que comparten progreso con un servidor y validan las coordenadas.
- Versión para otras ciudades: carga personajes y preguntas desde un archivo JSON y cambia el centro del juego (por ejemplo, San Miguel de Allende).
- Accesibilidad: agrega una lista de personajes sin cámara con la dirección y distancia leídas en voz alta.

🏅 INSIGNIA DESBLOQUEADA: Creador de juegos con realidad aumentada

Recuerda: la mejor realidad aumentada empuja a la gente a salir, caminar y descubrir su ciudad. ¡Que tu juego haga que Dolores Hidalgo se explore con curiosidad!